INFORMATION DISCLOSURE

INTRODUCTION

Information Disclosure is a critical security flaw that occurs when sensitive data is unintentionally revealed to unauthorized users. This data can include system files, credentials, database structure, server configuration, or source code. In penetration testing and bug bounty assessments, finding such leaks is essential, as attackers can use this information to exploit vulnerabilities. Through this report and hands-on labs, I explored the various forms and risks of information disclosure to strengthen my practical understanding and defensive capabilities.

1. What Is Information Disclosure?

- Information Disclosure occurs when sensitive data is unintentionally exposed to unauthorized parties.
- This can include error messages, source code, internal ports, confidential files, API keys, or configuration details.
- Even seemingly harmless metadata can help attackers plan further exploitation.
- In short, it's a leakage of internal information that attackers can use to compromise systems.

2. Types of Information Disclosure & Mitigations

Type	Example	Mitigation
Error messages/details	Stack traces, database errors	Use generic error messages; sanitize exceptions
Source code or comments	HTML comments revealing logic	Remove comments and minimize client-side scripts
Old APIs or debug endpoints	/api/v1/test, /debug/	Disable in production; implement strict access rules

Type	Example	Mitigation
Directory listings	index of /backup/	Use Options -Indexes or proper file access control
Server fingerprinting	Server version banners, HTTP headers	Remove or change server banners; use security headers
Configuration leaks	.git, .env, or database passwords	Use environment variables; ensure sensitive files are not public

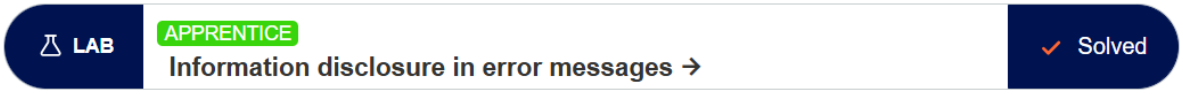
3. Tools & Techniques to Find Sensitive Data

- Burp Suite: Repeater, Scanner, and Discovery tools to identify leaks.
- DirBuster / Gobuster: Brute force hidden directories and files.
- Nikto: Web server scanner for misconfigurations.
- Source code analysis: Downloading .js, .css, .env files looking for secrets.
- Fuzzers: Test common endpoints for internal APIs or debug paths.
- Chrome/Firefox Developer Tools: Inspect network/tab headers for hidden data.

LAB COMPLETION

- I completed the PortSwigger lab on Information Disclosure:
  🔗 https://portswigger.net/web-security/all-labs#information-disclosure

Lab 1: Information Disclosure in Error Messages



- Revealed sensitive server details through verbose error responses, useful for attackers in further exploitation.

Solution:

1. *With Burp running, open one of the product pages.*
2. *In Burp, go to "Proxy" > "HTTP history" and notice that the GET request for product pages contains a productID parameter. Send the GET /product?productId=1 request to Burp Repeater. Note that your productId might be different depending on which product page you loaded.*
3. *In Burp Repeater, change the value of the productId parameter to a non-integer data type, such as a string. Send the request:*

   GET /product?productId="example"
4. *The unexpected data type causes an exception, and a full stack trace is displayed in the response. This reveals that the lab is using Apache Struts 2 2.3.31.*

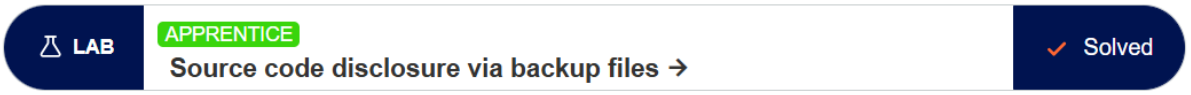
Lab 2: Information Disclosure on Debug Page

- Exposed internal debug information that should be restricted, including environment variables and dev tools.

Solution:

1. *With Burp running, browse to the home page.*
2. *Go to the "Target" > "Site Map" tab. Right-click on the top-level entry for the lab and select "Engagement tools" > "Find comments". Notice that the home page contains an HTML comment that contains a link called "Debug". This points to /cgi-bin/phpinfo.php.*
3. *In the site map, right-click on the entry for /cgi-bin/phpinfo.php and select "Send to Repeater".*
4. *In Burp Repeater, send the request to retrieve the file. Notice that it reveals various debugging information, including the SECRET_KEY environment variable.*

Lab 3: Source Code Disclosure via Backup Files

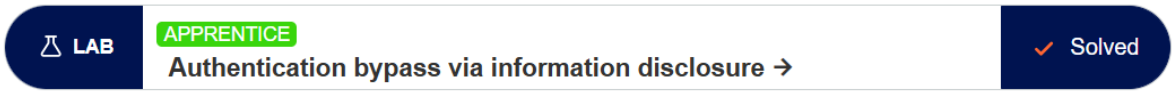


- Found accessible backup files like **.bak** containing full source code and hardcoded credentials.

Solution:

1. *Browse to /robots.txt and notice that it reveals the existence of a /backup directory. Browse to /backup to find the file ProductTemplate.java.bak. Alternatively, right-click on the lab in the site map and go to "Engagement tools" > "Discover content". Then, launch a content discovery session to discover the /backup directory and its contents.*
2. *Browse to /backup/ProductTemplate.java.bak to access the source code.*
3. *In the source code, notice that the connection builder contains the hard-coded password for a Postgres database.*

Lab 4: Authentication Bypass via Information Disclosure



- Used leaked credentials or known internal paths to bypass login and access restricted accounts.

Solution:

1. *In Burp Repeater, browse to GET /admin. The response discloses that the admin panel is only accessible if logged in as an administrator, or if requested from a local IP.*
2. *Send the request again, but this time use the TRACE method:*

   TRACE /admin

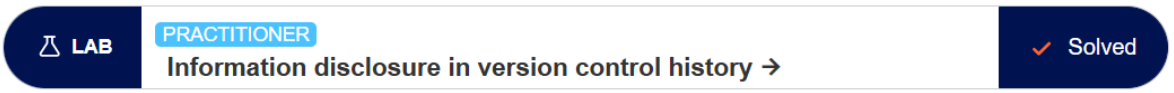
3. *Study the response. Notice that the X-Custom-IP-Authorization header, containing your IP address, was automatically appended to your*

request. This is used to determine whether or not the request came from the localhost IP address.

4. *Go to Proxy > Match and replace.*
5. *Under HTTP match and replace rules, click Add. The Add match/replace rule dialog opens.*
6. *Leave the Match field empty.*
7. *Under Type, make sure that Request header is selected.*
8. *In the Replace field, enter the following:*

   X-Custom-IP-Authorization: 127.0.0.1
9. *Click Test.*
10. *Under Auto-modified request, notice that Burp has added the X-Custom-IP-Authorization header to the modified request.*
11. *Click OK. Burp Proxy now adds the X-Custom-IP-Authorization header to every request you send.*
12. *Browse to the home page. Notice that you now have access to the admin panel, where you can delete carlos.*

Lab 5: Information Disclosure in Version Control History



- Extracted secrets from **.git** history where sensitive data wasn't properly removed before deployment.

Solution:

1. *Open the lab and browse to **/.git** to reveal the lab's Git version control data.*
2. *Download a copy of this entire directory. For Linux users, the easiest way to do this is using the command:*

wget -r https://YOUR-LAB-ID.web-security-academy.net/.git/

3. *Explore the downloaded directory using your local Git installation. Notice that there is a commit with the message "Remove admin password from config".*
4. *Look closer at the diff for the changed admin.conf file. Notice that the commit replaced the hard-coded admin password with an environment variable ADMIN_PASSWORD instead. However, the hard-coded password is still clearly visible in the diff.*
5. *Go back to the lab and log in to the administrator account using the leaked password.*
6. *To solve the lab, open the admin interface and delete carlos.*

CONCLUSION

Information disclosure, though often overlooked, can be the root cause of major breaches. It provides attackers with valuable insight into the target system, enabling more targeted and effective attacks. Through tools like Burp Suite, DirBuster, and manual analysis, security professionals can proactively detect and mitigate these leaks. Completing the PortSwigger labs gave me a real-world understanding of how critical these exposures can be and how to guard against them in live environments.